

Introduction to the .NET Platform and C#

Learning Objectives

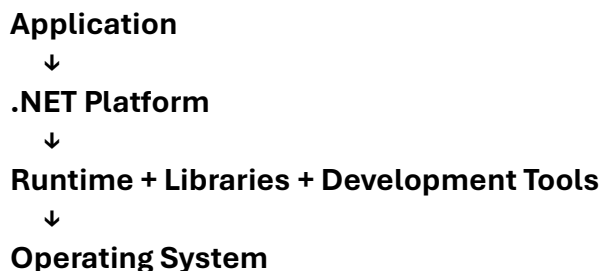
By the end of this section, you should be able to:

- Explain the purpose of the .NET platform.
 - Distinguish between the historical **.NET Framework**, **.NET Core**, and the modern **.NET** platform.
 - Identify important technologies in the .NET ecosystem, including C#, ASP.NET, Web API, and ADO.NET.
 - Describe the basic structure and organization of a C# program.
 - Use namespaces and using directives.
 - Define a basic C# class containing fields, constructors, and methods.
 - Distinguish between **value types** and **reference types**.
 - Identify and use common predefined C# types.
 - Explain the **.NET Unified Type System**.
 - Explain and perform basic **boxing and unboxing** operations.
 - Explain how boxing and unboxing occur when using non-generic collections such as ArrayList.
 - Use Console.WriteLine() to display and format basic program output.
-

1. What Is .NET?

.NET is Microsoft's software development platform for building and running applications.

A useful conceptual view is:



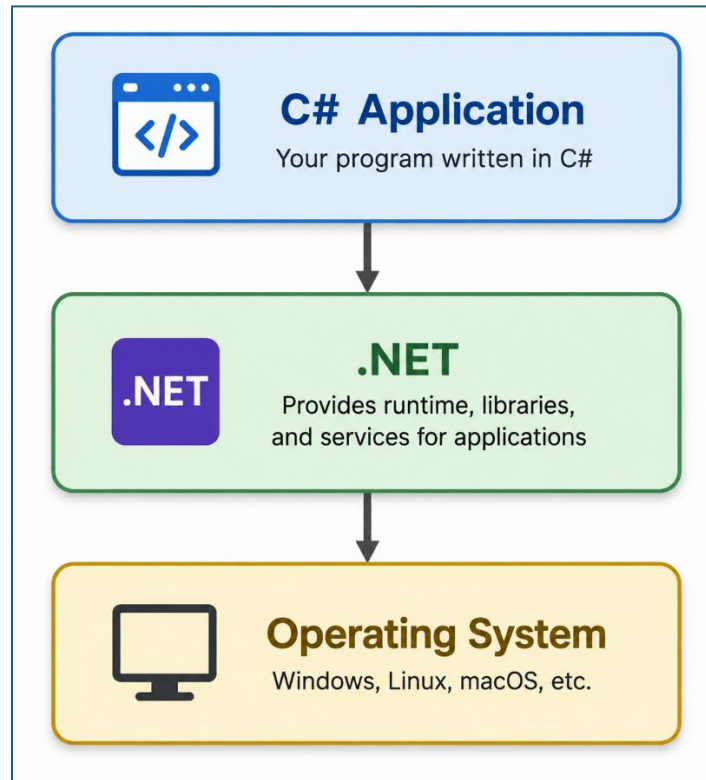
Rather than requiring developers to implement common functionality themselves, .NET provides a large collection of reusable libraries and runtime services.

C# is one of the primary programming languages used to develop .NET applications.

NOTE — C# and .NET Are Not the Same Thing

C# is a programming language.

.NET is the development platform on which C# applications can be built and executed.



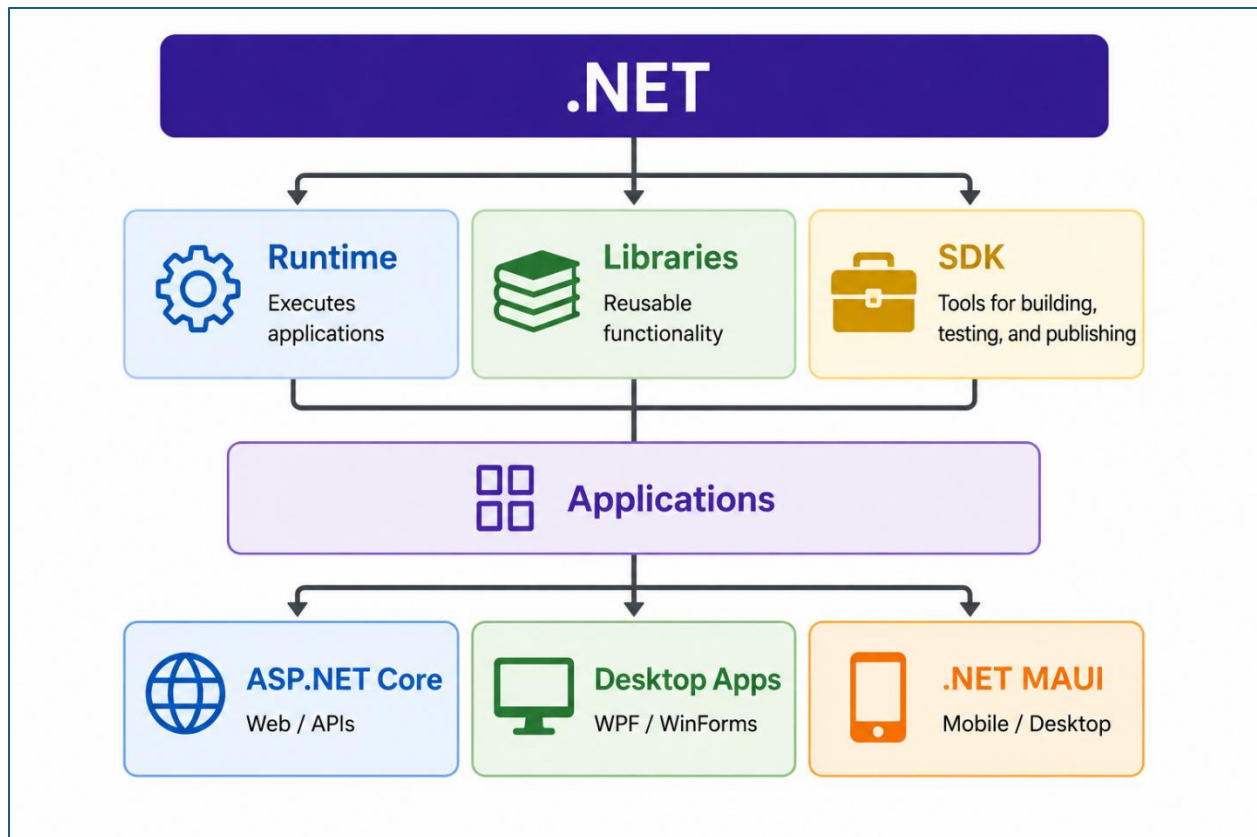
2. The Modern .NET Ecosystem

.NET is a unified, open-source, cross-platform development platform for building many types of applications.

The platform provides several fundamental components:

- the **.NET Runtime**, which executes application code;
- the **.NET libraries**, which provide reusable functionality;
- the **.NET SDK**, which provides tools for creating, building, testing, and publishing applications;
- programming languages such as **C#**;
- application frameworks such as **ASP.NET Core**, **Windows Forms**, **WPF**, and **.NET MAUI**.

A useful conceptual view is:



The same .NET platform can therefore support many different application types.

Web and Backend Development

ASP.NET Core is the primary framework for building web applications and backend services with .NET.

It can be used to create:

- web applications;
- REST APIs;
- backend services;
- cloud applications;
- microservices.

ASP.NET Core applications can run on Windows, Linux, and macOS.

Desktop Development

.NET supports Windows desktop development through frameworks such as:

- **Windows Forms (WinForms);**

- **Windows Presentation Foundation (WPF).**

These frameworks are commonly used to create graphical Windows desktop applications.

Cross-Platform Applications

.NET MAUI provides a framework for building native mobile and desktop applications using C# and .NET.

A .NET MAUI application can target platforms including:

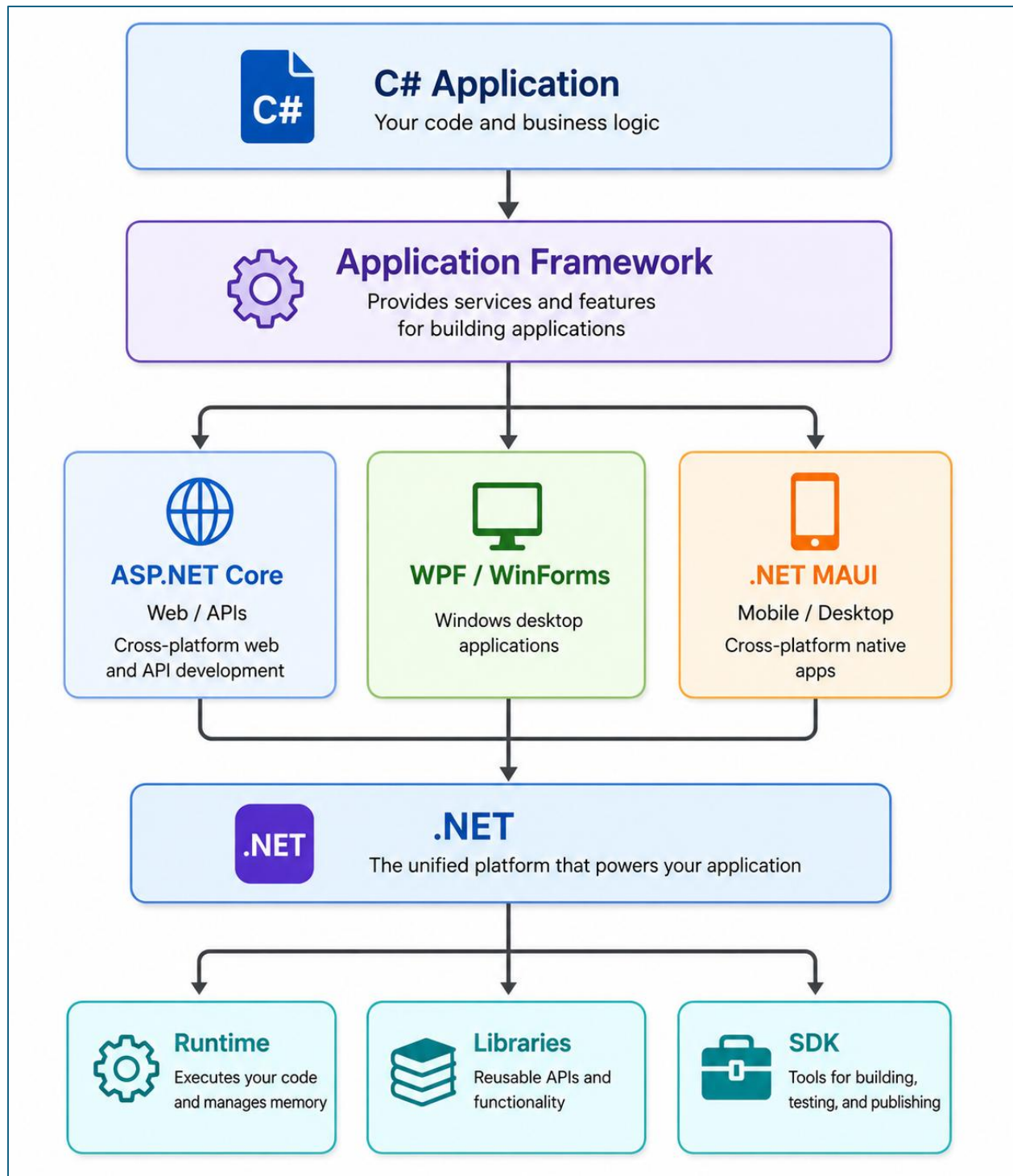
- Android;
- iOS;
- Windows;
- macOS.

This allows developers to share significant portions of their code across multiple platforms.

3. Core Components of .NET

Instead of choosing between separate .NET platforms, development generally starts with **.NET** and then selects the appropriate application framework or workload.

A simplified architecture is:



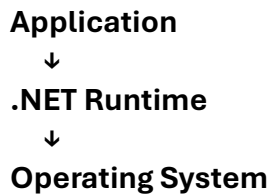
.NET Runtime

The **.NET Runtime** provides the environment in which .NET applications execute.

It provides services such as:

- execution of managed code;
- automatic memory management;
- garbage collection;
- exception handling;
- type safety.

Conceptually:



.NET Libraries

.NET includes a large collection of reusable libraries.

These libraries provide functionality for common programming tasks such as:

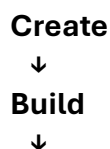
- collections;
- file operations;
- networking;
- text processing;
- JSON processing;
- date and time operations;
- asynchronous programming.

For example, classes such as String, List<T>, File, and HttpClient are provided through .NET libraries.

.NET SDK

The **.NET SDK (Software Development Kit)** contains the tools developers need to create and work with .NET applications.

The SDK supports activities such as:



Test
↓
Run
↓
Publish

The dotnet command provides access to many of these development operations from the command line.

For example:

```
dotnet new  
dotnet build  
dotnet run  
dotnet test  
dotnet publish
```

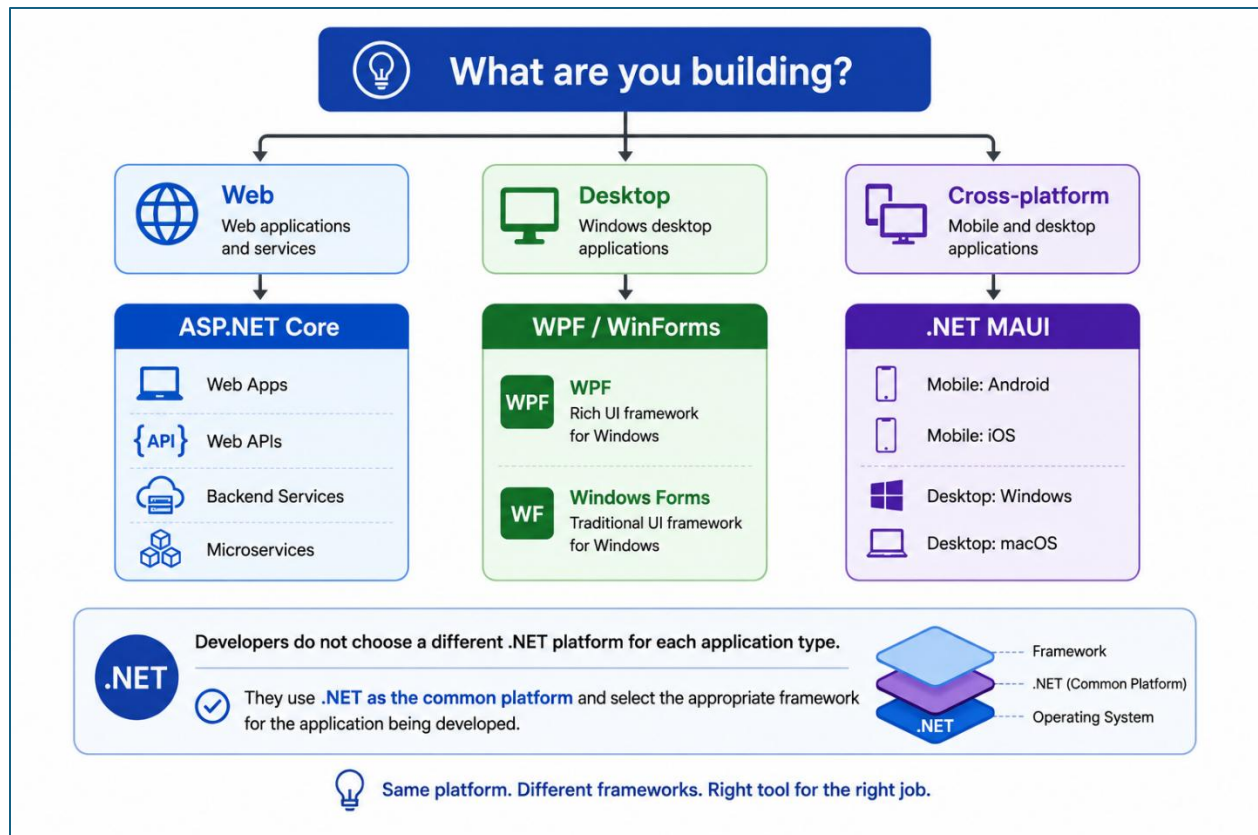
4. Choosing a .NET Application Framework

In .NET development, the primary decision is generally **what type of application you want to build**.

The application type determines which .NET framework or technology is most appropriate.

Application Type	Common Technology
Console application	.NET
Web application	ASP.NET Core
REST API	ASP.NET Core Web API
Backend service	.NET / ASP.NET Core
Cloud application	.NET / ASP.NET Core
Microservice	.NET / ASP.NET Core
Windows desktop application	WPF / Windows Forms
Cross-platform mobile/desktop application	.NET MAUI

A useful decision model is:



Therefore, developers generally do not select a different ".NET platform" for each application type. Instead, they use **.NET as the common platform** and select the appropriate framework for the application being developed.

TIP

Before starting a project, identify the application's target platform and workload. This helps determine the appropriate .NET framework, project template, dependencies, and deployment approach.

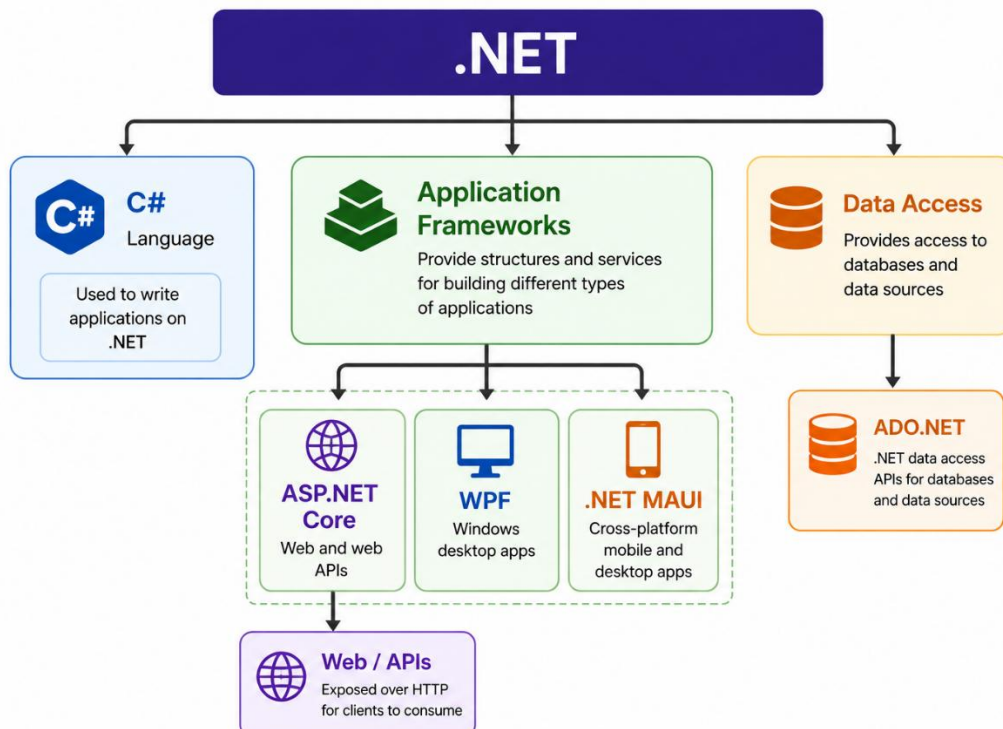
5. Important .NET Terminology

Several terms appear repeatedly throughout .NET development.

Term	Meaning
.NET	The development platform containing the runtime, libraries, SDK, tools, and application frameworks used to build and run applications.
C#	A major programming language used to develop .NET applications.

.NET Runtime	The execution environment responsible for running .NET application code and providing services such as memory management and garbage collection.
.NET SDK	The tools and libraries used to create, build, test, run, and publish .NET applications.
ASP.NET Core	The .NET framework for developing web applications, Web APIs, backend services, and related server-side applications.
Web API	An HTTP-based interface that allows applications and services to communicate over a network.
.NET MAUI	A .NET framework for developing cross-platform native mobile and desktop applications.
WPF	A framework for developing Windows desktop applications with rich graphical user interfaces.
Windows Forms	A .NET framework for developing traditional Windows desktop applications.
ADO.NET	.NET APIs for working directly with databases and other data sources.

These technologies occupy different roles within the ecosystem:



PITFALL — Mixing Language, Platform, and Framework Terminology

These terms should not be used interchangeably.

C# is a programming language.

.NET is the development platform.

ASP.NET Core is a web framework that runs on .NET.

Web API describes an HTTP-based interface that applications can expose or consume.

ADO.NET provides APIs for data access.

A developer can therefore use **C# with ASP.NET Core on .NET** to build a web application or Web API.

6. Introduction to C#

C# is a strongly typed programming language designed for the .NET platform.

It supports several programming styles, including:

- object-oriented programming;
- functional programming features;
- generic programming;
- asynchronous programming;
- pattern matching;
- declarative programming constructs.

C# emphasizes:

- concise and readable syntax;
- strong compile-time type checking;
- automatic memory management;
- type inference;
- null safety;
- asynchronous programming;
- reusable components;
- expressive data modeling.

A simple C# program can contain only:

```
Console.WriteLine("Hello, World!");
```

This is a complete C# console application.

There is no requirement to explicitly write a Program class or a **Main()** method for a basic console application.

The compiler generates the necessary entry point automatically.

7. Top-Level Statements

C# supports **top-level statements**.

They allow executable statements to appear directly in a source file without explicitly declaring a **Main()** method.

For example:

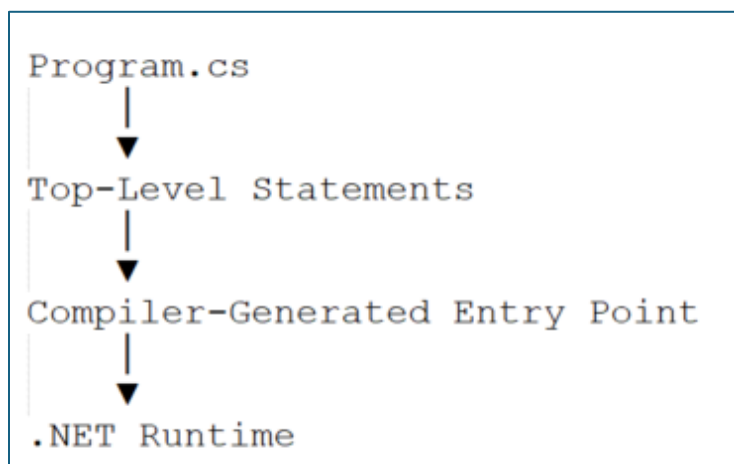
```
string name = "Alice";

Console.WriteLine($"Hello, {name}!");
```

This program:

1. creates a string variable;
2. displays a message;
3. uses string interpolation to insert the variable into the output.

Conceptually:



Top-level statements reduce unnecessary boilerplate and are the default style generated for new .NET console applications.

Command-Line Arguments

Command-line arguments remain available through the automatically provided `args` variable.

For example:

```
Console.WriteLine($"Number of arguments: {args.Length}");
```

There is no need to explicitly declare:

```
Main(string[] args)
```

to access command-line arguments.

8. Implicit Using Directives

Many introductory C# examples traditionally begin with:

```
using System;
```

In .NET projects, common namespaces are often imported automatically through **implicit using directives**.

Therefore:

```
Console.WriteLine("Hello");
```

normally works without explicitly writing:

```
using System;
```

The .NET SDK determines which namespaces are implicitly available based on the project type.

For example, console applications commonly have access to namespaces containing frequently used types for:

- basic system functionality;
- collections;
- file operations;
- LINQ;
- networking;

- threading;
- asynchronous tasks.

Explicit Using Directives

A using directive is still required when a namespace is not already available.

For example:

```
using System.Text;

StringBuilder builder = new();

builder.AppendLine("Hello");
```

The **using** directive allows types from the specified namespace to be referenced without their fully qualified names.

Without it:

```
System.Text.StringBuilder builder = new();
```

Global Using Directives

C# also supports **global using directives**.

For example:

```
global using System.Text;

global using System.Text.Json;
```

A global **using** applies throughout the project rather than only to one source file.

9. Namespace Declarations

Namespaces organize related types and help prevent naming conflicts.

C# supports **file-scoped namespaces**.

For example:

```
namespace MyApplication.Models;

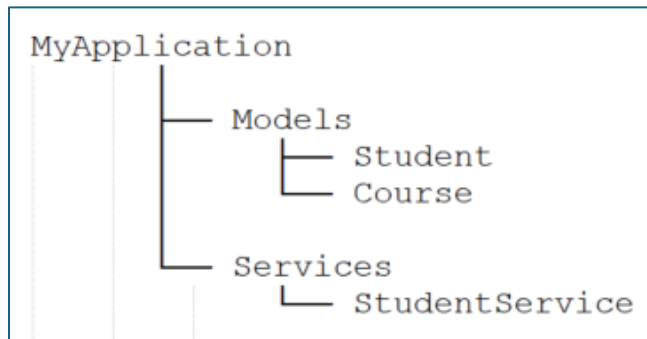
public class Student
```

```
{  
}
```

The namespace applies to the remainder of the file.

This avoids additional braces and indentation.

Conceptually:



A common convention is for namespace names to reflect the organization of the project.

For example:

MyApplication.Models

MyApplication.Services

MyApplication.Data

MyApplication.Controllers

TIP — Prefer File-Scoped Namespaces

When all types in a file belong to the same namespace, file-scoped namespaces provide a cleaner structure:

```
namespace MyApplication.Models;
```

```
public class Student
```

```
{  
}
```

rather than adding another level of indentation around the entire file.

10. Classes in C#

Classes remain one of the fundamental mechanisms for defining reference types in C#.

Consider:

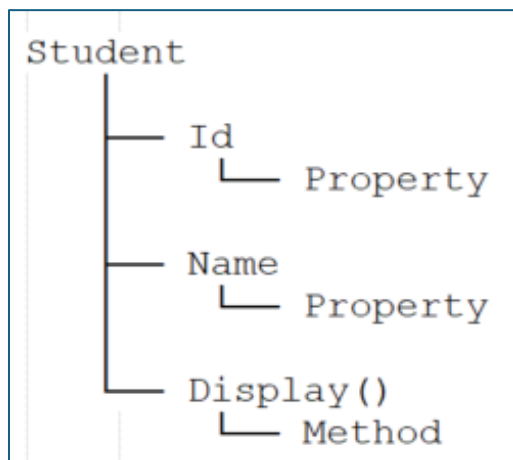
```
namespace MyApplication.Models;

public class Student
{
    public int Id { get; set; }

    public string Name { get; set; } = "";

    public void Display()
    {
        Console.WriteLine($"{Id}: {Name}");
    }
}
```

The class contains:



C# applications frequently expose data through **properties** rather than public fields.

For example:

```
public int Id { get; set; }
```

defines an automatically implemented property.

11. Object Creation and Target-Typed new

Objects can be created using the new operator.

For example:

```
Student student = new Student();
```

C# also supports **target-typed new** when the type is already clear from the declaration:

```
Student student = new();
```

Both create a Student object.

The second form avoids repeating the type name unnecessarily.

For example:

```
List<string> names = new();
```

instead of:

```
List<string> names = new List<string>();
```

12. Type Inference with var

C# is statically typed, but the compiler can often infer a local variable's type.

For example:

```
var age = 25;  
var name = "Alice";  
var price = 19.99;
```

The compiler determines:

```
age      → int  
name     → string  
price    → double
```

`var` does **not** make C# dynamically typed.

The variables still have fixed compile-time types.

For example:

```
var number = 10;
```

is effectively inferred as:

```
int number = 10;
```

After the type has been inferred, it cannot arbitrarily change:

```
var number = 10;  
  
// Invalid:  
// number = "Hello";
```

TIP

Use var when the type is obvious from the expression or when it improves readability.

Do not use it merely to hide important type information.

13. C# Source Files and Project Organization

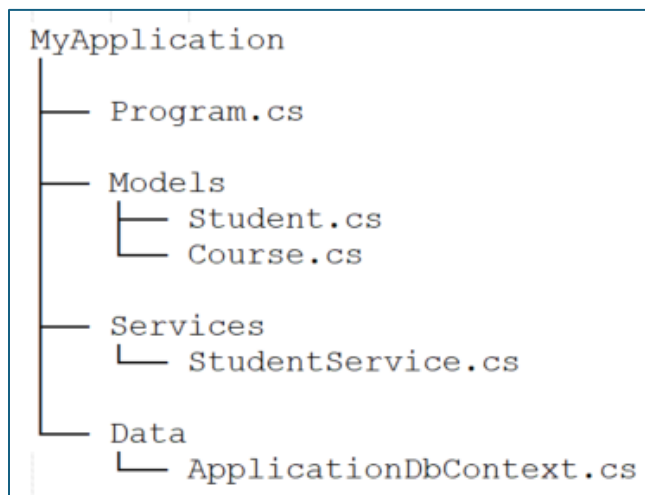
C# source files use the extension:

.cs

For example:

Program.cs
Student.cs
Course.cs
StudentService.cs

A project might be organized as:



Each source file can contain type declarations such as:

- classes;
- records;
- structs;
- interfaces;
- enums.

A common convention is to keep important types in separate files so that the project remains easy to navigate.

14. Comments and Documentation Comments

Single-line comments begin with:

```
// Calculate the final grade
```

Multi-line comments use:

```
/*  
    This comment can  
    span multiple lines.  
*/
```

Documentation comments begin with:

```
///
```

For example:

```
/// <summary>  
/// Represents a student.  
/// </summary>  
public class Student  
{  
}
```

Documentation comments can be processed by development tools to provide structured API documentation and IntelliSense information.

TIP

Comments should generally explain **why** something is being done rather than merely restating obvious code.

15. The C# Type System

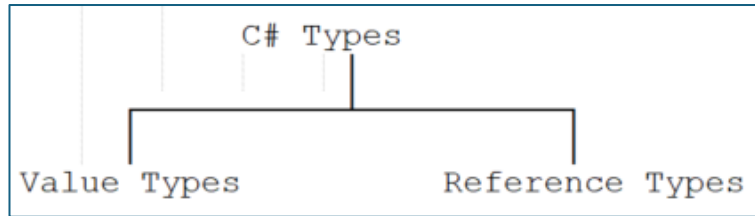
C# is a **statically typed** language.

Every variable and expression has a type.

A type determines:

- what values can be represented;
- which operations are valid;
- which members are available;
- how values behave during assignment and method calls.

At a fundamental level:



Understanding this distinction remains fundamental in C#.

16. Value Types

A value-type variable represents its value directly.

Examples include:

```
int age = 25;
double temperature = 21.5;
bool active = true;
decimal price = 19.99m;
```

Common value types include:

- numeric types;
- bool;
- char;
- enums;
- structs.

For example:

```
enum Status
{
    Active,
    Inactive
}
```

and:

```
struct Point
{
    public int X { get; set; }
    public int Y { get; set; }
}
```

When one value-type variable is assigned to another, its value is copied.

For example:

```
int first = 10;
int second = first;

second = 20;

Console.WriteLine(first); // 10
Console.WriteLine(second); // 20
```

Changing second does not change first.

17. Reference Types

Reference types include:

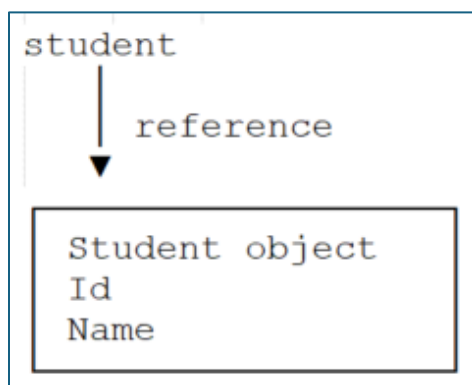
- classes;
- arrays;
- interfaces;
- delegates;
- strings;
- records declared as record class.

For example:

```
Student student = new();
```

The variable student refers to a Student object.

A simplified conceptual model is:



Reference semantics become important when multiple variables refer to the same object.

For example:

```
Student first = new()
{
    Id = 1,
    Name = "Alice"
};

Student second = first;

second.Name = "Bob";

Console.WriteLine(first.Name);
```

The output is:

Bob

because both variables refer to the same object.

18. Value Types vs. Reference Types

A useful introductory comparison is:

Value Types	Reference Types
Represent their value directly	Refer to objects
Assignment copies the value	Assignment copies the reference
Examples: int, double, bool, structs, enums	Examples: classes, arrays, interfaces, string
Non-nullable by default	Can participate in nullable reference type analysis

Conceptually:

VALUE TYPE

```
int first = 10;
```

first



REFERENCE TYPE

```
Student student = new();
```

student $\longrightarrow$ Student Object

PITFALL — Value Type Does Not Mean Primitive

Structs and enums are also value types.

Therefore:

Value Type $\neq$ Primitive Type

The distinction is based on the semantics of the type, not simply whether it appears to be a simple built-in type.

19. Predefined C# Types

C# provides keywords for commonly used .NET types.

Integer Types

- byte
- sbyte
- short
- ushort
- int
- uint
- long
- ulong

Floating-Point and Decimal Types

- float

- double
- decimal

Other Common Types

- bool
- char
- string
- object

These C# keywords correspond to .NET types.

For example:

int	→ System.Int32
long	→ System.Int64
double	→ System.Double
bool	→ System.Boolean
string	→ System.String
object	→ System.Object

Therefore:

```
int number = 10;
```

and:

```
System.Int32 number = 10;
```

refer to the same underlying type.

In normal C# code, the language keywords such as int, string, and bool are generally preferred.

20. Nullable Reference Types

C# provides **nullable reference type analysis** to help detect possible null-reference problems during compilation.

Consider:

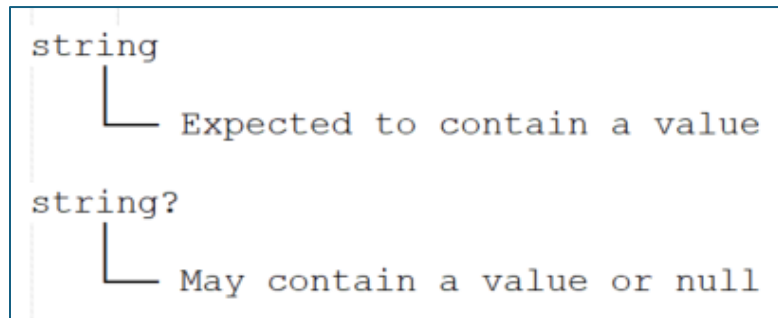
```
string name = "Alice";
```

This indicates that name is expected to contain a string.

If a reference is intentionally allowed to be null, it can be declared using ?:

```
string? middleName = null;
```

Conceptually:



For example:

```
string? nickname = null;
if (nickname is not null)
{
    Console.WriteLine(nickname.Length);
}
```

The null check allows the compiler to determine that nickname is safe to use inside the if statement.

Why This Matters

Null-reference errors are a common source of runtime failures.

Nullable reference types move many potential null problems from **Runtime** to **Compile-Time Analysis** allowing developers to identify them earlier.

21. The Unified .NET Type System

C# and .NET use a unified type system.

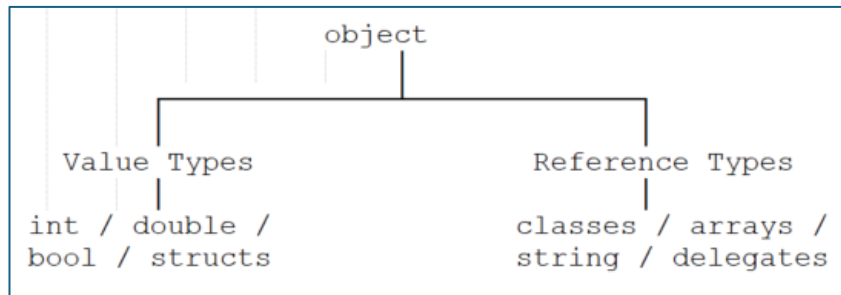
All .NET types ultimately relate to:

System.Object

represented in C# by:

object

A simplified conceptual view is:



This unified type system allows APIs and language features to work consistently across many different types.

22. Boxing and Unboxing

Boxing and unboxing are still part of C# and should be understood, but they should not be presented as the normal way to build collections.

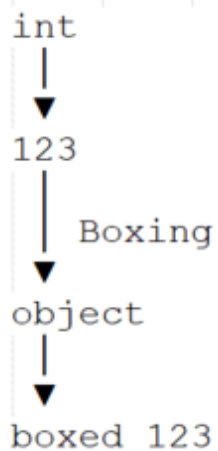
Boxing occurs when a value type is converted to object or an appropriate interface type.

For example:

```
int number = 123;
```

```
object value = number;
```

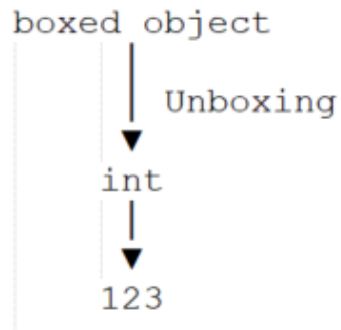
Conceptually:



Unboxing extracts the value:

```
int restored = (int)value;
```

Conceptually:



Boxing can involve additional allocation and runtime work.

For this reason, C# APIs generally use **generics** when type-specific storage is required.

23. Generic Collections

C# applications commonly use **generic collections**.

For example, to store integers:

```
List<int> numbers = new();
```

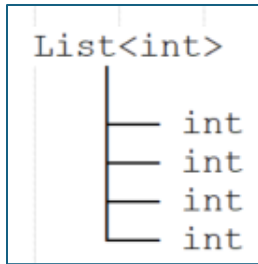
Values can then be added directly:

```
for (int x = 0; x < 10; x++)
{
    numbers.Add(x);
}
```

And retrieved without casting:

```
int value = numbers[5];
```

Conceptually:



The collection knows that its elements are integers.

This provides:

- compile-time type safety;
- clearer code;
- no manual casting when retrieving values;
- avoidance of unnecessary boxing for value-type elements in common generic collection scenarios.

For example:

```
List<string> names = new()  
{  
    "Alice",  
    "Bob",  
    "Charlie"  
};
```

The compiler prevents incompatible values from being inserted:

```
// Invalid:  
// names.Add(100);
```

PITFALL — Do Not Use ArrayList for New Code

Older C# materials often demonstrate collections using `ArrayList`.

applications should normally use generic collections such as:

`List<T>`

`Dictionary<TKey, TValue>`

`HashSet<T>`

`Queue<T>`

Stack<T>

Generic collections provide stronger type safety and avoid many of the casting and boxing issues associated with older object-based collections.

24. Collection Expressions

C# supports concise **collection expressions**.

For example:

```
List<string> names = ["Alice", "Bob", "Charlie"];
```

An array can similarly be initialized as:

```
int[] scores = [85, 90, 78, 92];
```

This provides a concise syntax for creating collections from known values.

For example:

```
List<int> numbers = [10, 20, 30, 40];

foreach (int number in numbers)
{
    Console.WriteLine(number);
}
```

25. String Interpolation

C# commonly uses **string interpolation** to construct formatted strings.

Instead of concatenating strings:

```
string name = "Alice";
```

```
int age = 25;
```

```
Console.WriteLine("Name: " + name + ", Age: " + age);
```

use:

```
Console.WriteLine($"Name: {name}, Age: {age}");
```

The **\$** indicates an interpolated string.

Expressions can be placed directly inside **{ }**.

For example:

```
int x = 10;
int y = 20;

Console.WriteLine($"{x} + {y} = {x + y}");
```

Output:

10 + 20 = 30

String interpolation is generally clearer and easier to maintain than manual string concatenation for formatted output.

26. Pattern Matching and Null Checks

C# makes extensive use of **pattern matching**.

A simple example is checking whether a reference is not null:

if (name is not null)

```
if (name is not null)
{
    Console.WriteLine(name);
}
```

Type patterns can also combine type checking and variable creation:

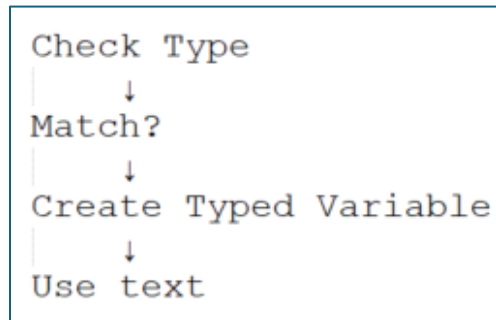
```
object value = "Hello";

if (value is string text)
{
    Console.WriteLine(text.Length);
}
```

Here:

value is string text

does three things conceptually:



Pattern matching becomes increasingly useful in C# for working with object models, APIs, and conditional logic.

27. C# Program Organization

C# applications should not be thought of only as:

Namespaces



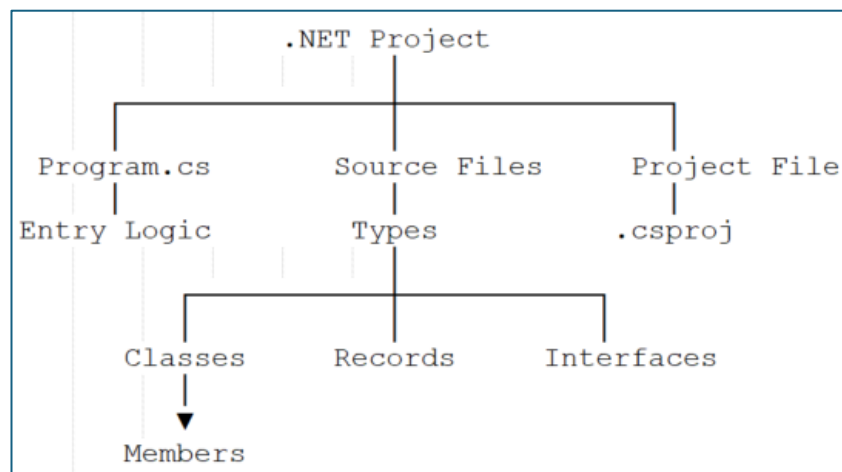
Types



Members

That model remains conceptually useful for understanding declarations, but applications also include top-level code, project configuration, dependencies, and multiple organizational layers.

A more practical introductory model is:



The project file:

.csproj

defines important project information such as:

- the target .NET framework;
- project type;
- compiler settings;
- package dependencies.

The source files contain the application's executable logic and type definitions.

28. Putting the Concepts Together

The concepts introduced in this section provide a foundation for C# development.

A useful mental model is:

